

Faillle CSRF

Norguet Enzo

Définition : Une vulnérabilité CSRF (pour Cross-Site Request Forgery, en français : falsification de requête inter-site) est une faille qui permet à un attaquant d'abuser à la fois d'un utilisateur, d'un navigateur web et d'un serveur. Le principe est assez simple. L'objectif de l'attaque est de forcer un utilisateur authentifié sur un site ou une application web à exécuter des actions spécifiques à son insu.

Son fonctionnement : Une attaque CSRF repose sur l'utilisation de ingénierie sociale. Un assaillant trompe sa victime en envoyant un lien par chat ou e-mail. Lorsqu'une victime est un utilisateur sans privilèges d'administration, l'attaque CSRF peut l'inciter à faire des choses telles que modifier une adresse e-mail telle qu'elle apparaît dans le système du site cible, transférer des fonds d'un compte, modifier les informations de nom d'utilisateur, etc. Si la victime dispose de privilèges d'administrateur, l'attaque CSRF peut être utilisée pour modifier la fonction de l'application Web elle-même.

Les attaques CSRF se concentrent sur les fonctions qui modifient le serveur d'une application Web cible. La façon dont un attaquant exploite les changements mis en œuvre par l'attaque varie en fonction de ses objectifs. Ils peuvent vouloir voler de l'argent immédiatement et là. Ou ils peuvent vouloir attendre une date ultérieure pour modifier les informations sur le serveur. L'attaquant peut également modifier les informations du compte, puis vendre les identifiants à un acheteur.

Parfois, le script d'attaque CSRF peut être stocké sur le site cible. Dans le codage HTML (Hypertext Markup Language), un fichier IMG est utilisé pour stocker des images et une balise iframe est utilisée pour placer un document dans la page. Si un attaquant peut accéder au code HTML du site, il peut intégrer le code d'attaque CSRF dans un fichier IMG ou une balise iframe. Ces types de vulnérabilités sont connus sous le nom de failles CSRF stockées. Ce type d'attaque peut également être réalisé grâce à des méthodes XSS plus complexes.

Méthode pour lutter contre : **Utiliser un jeton ou un token pour contrer les attaques CSRF**

Les jetons, ou token en anglais, permettent de sécuriser la relation application web, navigateur et utilisateurs. Cela fonctionne de la façon suivante. Lors de chaque cession, un site sécurisé attribue un jeton CSRF à l'utilisateur. Ce token est partagé entre le site et le navigateur de l'internaute. En pareil cas, quand le navigateur lance une requête, le serveur vérifie la présence de ce jeton. S'il n'y est pas ou si le token est erroné, le serveur ne répondra pas à la requête.

Se protéger des attaques CSRF en utilisant l'attribut Samesite sur les cookies

L'attribut Samesite est une entité qu'une application va intégrer à un cookie de session afin de renforcer l'authentification de l'utilisateur. Or, lors d'une attaque CSFR, le pirate ne

dispose pas de cet élément. Le défaut de cet attribut Samesite va signaler au serveur que la requête n'est pas légitime. Le serveur de l'application visée va alors refuser d'exécuter les requêtes qui ne le comportent pas.

L'utilisation de l'en-tête Referer pour se prémunir des requêtes de types CSRF

Une autre méthode de protection contre une attaque de type CSRF est l'utilisation de l'en-tête referer. Il faut savoir que dans une requête HTTP se trouve un champ qui indique d'où vient la requête. C'est la zone referer. Elle contient l'URL et permet au serveur d'identifier d'où vient une requête. Cet outil permet d'identifier la page visitée précédente. C'est le dispositif qui permet de revenir à la page précédente d'un seul clic. Cette fonction permet de vérifier que les requêtes viennent bien d'un site qui est légitime.

La double authentification : l'arme absolue contre les attaques de type CSFR

La double authentification permet à un site de se faire confirmer l'identité de l'internaute désireux de se connecter à son compte. Lorsque celui-ci cherche à se connecter à son compte, le site lui fait parvenir un SMS sur son téléphone mobile. L'internaute entre alors le code réceptionné dans le champ dédié. Il confirmera ainsi son identité. Suite à cette opération, il pourra accéder à son compte.

Les tentatives de fraudes CSRF sont malheureusement fréquentes. Elles ne sont possibles que sur les sites qui authentifient leurs utilisateurs par des cookies uniquement. Ainsi, le premier réflexe à avoir est de se déconnecter après chaque utilisation. Jeton, attribut Samesite, attribut referer forment de bonnes parades, mais l'arme absolue reste néanmoins la double authentification.

Code php commentés de cette méthode :

```
<?php
session_start(); // Démarre la session pour stocker le jeton CSRF

// Fonction pour générer un jeton CSRF sécurisé et le stocker en session
function generateCsrfToken() {
    // Si le jeton n'existe pas déjà en session, on le crée
    if (empty($_SESSION['csrf_token'])) {
        // Génère 32 octets aléatoires et les convertit en une chaîne hexadécimale
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
}

// Génère le jeton CSRF si nécessaire
generateCsrfToken();

// Traitement du formulaire lors de la soumission
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
```

```
// Vérifie que le jeton CSRF est présent dans la soumission et correspond à celui stocké en session
```

```
if (!isset($_POST['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {
```

```
    // Si le jeton n'est pas valide, on arrête l'exécution et affiche une erreur  
    die("Échec de la vérification CSRF : jeton invalide.");  
}
```

```
// Si le jeton est valide, on peut traiter les données du formulaire
```

```
// Par exemple, on récupère et sécurise le nom d'utilisateur envoyé
```

```
$username = htmlspecialchars($_POST['username']);
```

```
echo "Formulaire soumis avec succès par : " . $username;
```

```
exit; // Termine le script après traitement
```

```
}
```

```
?>
```

```
<!DOCTYPE html>
```

```
<html lang="fr">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Protection CSRF</title>
```

```
</head>
```

```
<body>
```

```
    <!-- Formulaire HTML incluant le jeton CSRF dans un champ caché -->
```

```
    <form method="POST">
```

```
        <!-- Injection du jeton CSRF dans le formulaire -->
```

```
        <input type="hidden" name="csrf_token" value="<?php echo  
htmlspecialchars($_SESSION['csrf_token']); ?>">
```

```
        <label for="username">Nom d'utilisateur :</label>
```

```
        <input type="text" name="username" id="username" required>
```

```
        <button type="submit">Envoyer</button>
```

```
    </form>
```

```
</body>
```

```
</html>
```